

Software Requirements Specification (SRS)

For: EduCampus Collaboration Platform

Course: Software Engineering II

Instructor: Dr. Ali Mehmood Khan

Team members:

Ada Alkim Acikyol 28576498

Frenkli Zeqollari 47765319

Ipek Zobu 30971700

Mehlika Rana Akbay 51259883

Date: 15 November 2025

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, and design requirements of the **EduCampus**, a community-driven academic web-based platform connecting students, developed for universities located in **Potsdam, Germany**.

The document provides a detailed description of the system's capabilities, user interactions, and operational environment. The main goal of this system is to **facilitate collaboration among university students**, enabling them to form groups, manage projects, communicate in real time, and share resources. This SRS is intended for:

- **Developers**, who will use it to implement the system according to specified requirements.
- **Testers**, who will verify compliance of the software with the requirements.
- **Project Managers**, who will plan, track, and ensure delivery of project milestones.

1.2 Scope

EduCampus is a **web-based collaborative system** designed for students from universities in Potsdam.

It allows students to:

- Upload, share, and discover academic resources (notes, summaries, exercises, etc.).
- Search and filter the content by subject, title, and university.
 - Communicate via integrated direct messages and group channels.
 - Students can report inappropriate content.
 - The platform focuses on **student-to-student (P2P) resource exchange** within Potsdam's university ecosystem and is moderated by admin users.

In Scope:

- Enable **peer-to-peer resource exchange** (notes, summaries, exercises).
- Foster academic collaboration among universities in Potsdam.
- Offer moderated communication and support to maintain quality and safety.
- Reduce administrative overhead for instructors.

Out of scope:

- Full LMS functionality (grading, assignments, course enrollment).
- Universities outside Potsdam (current version).
- Complex role hierarchy beyond Student/Admin.
- The platform is accessible only via modern browsers.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definitions
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
DBMS	Database Management System
API	Application Programming Interface
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
P2P	Peer-to-Peer
EduCampus	Potsdam student collaboration platform

1.4 References

- IEEE 830 / ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering Standards

- Software Engineering 2 – Session 5, Dr. Ali Mehmood Khan (2025)
- EduCampus Figma Design and Website (2025)

1.5 Overview

This document is divided into the following major sections:

- **Section 1:** Introduction of the document.
- **Section 2: Overall Description** - General system overview, product context, user types, and constraints.
- **Section 3: Specific Requirements** - Detailed description of all functional, interface, and non-functional requirements.
- **Section 4: Appendices** - Supporting material including glossary, mockups, and Assumptions.

2 Overall Description

2.1 Product Perspective

EduCampus is an **independent client server web application** with front-end and back-end components communicating over HTTPS.

- **Frontend:** React.js + Tailwind CSS
- **Backend:** Python (Flask)
- **Database:** PostgreSQL
- **Hosting:** Cloud-based environment (Render)
- **Communication:** HTTPS for all API calls, WSS (Secure WebSocket) for real-time chat

External dependencies: Email service for verification and notification; cloud storage for file uploads.

2.2 Product Functions

Function	Description
User Registration & Login	Students create verified accounts using university email and select their university (limited to Potsdam institutions)
Resource Upload & Sharing	Students upload, and categorize notes or documents; others can view, or download

Search & Filter	Users search by title, subject, or university
Chat System	Peer-to-Peer messaging and group discussion channels
Moderation & Reporting	Students can report inappropriate content; admins review and act
User Profiles & Reputation	Displays uploaded resources and basic user information.

2.3 User Classes and Characteristics

User Type	Description	Technical Skill
Student (User)	Primary user; uploads and downloads resources, joins chats, uses Help Center	Basic
Admin (Moderator)	Verifies users, handles reports	Intermediate - Advanced

2.4 Operating Environment

Component	Description
Frontend	React.js + Tailwind CSS
Backend	Python (Flask)
Database	PostgreSQL
Browsers	Chrome, Firefox, Safari, Edge
OS Compatibility	Windows, macOS, Linux
Network	Stable internet connection via HTTPS

2.5 Design and Implementation Constraints

- Must comply with GDPR standards.
- Supports ≥ 500 concurrent users.
- Response time ≤ 3 seconds.
- File uploads ≤ 100 MB per resource.
- Development within a 6-month semester timeline.

2.6 User Documentation

- Integrated Help Center and FAQ.
- “Getting Started” guide for first-time users.
- PDF-based **User Manual** for students.

2.7 Assumptions and Dependencies

- Users have valid university email accounts.
- Admins actively moderate reported content.
- Hosting and database services remain available.
- Email server integration is functional for notifications.

3 Specific Requirements

3.1 Functional Requirements

FR-1: User Authentication

Description:

The system must allow users to register and log in using university email credentials.

Scenario:

1. A new student opens the platform and clicks “*Register.*”
2. The system requests the student’s university email.
3. The student sets a password and profile details.
4. Upon login, the system redirects to the dashboard.

Exceptions:

- Invalid email → error message “Unrecognized university domain.”
- Duplicate account → system rejects registration.

FR-2: Resource Upload

Description:

The system must allow users to upload study files (PDF, DOCX) with relevant metadata.

Scenario:

1. The user navigates to “**Upload Resource.**”

2. The system requests the file, title, and subject.
3. The user submits the upload form.
4. The system validates file type and size.
5. The file is stored and made available for all verified users.

Exceptions:

- Invalid file format → System displays: *“Unsupported file type.”*
- File exceeds size limit → Upload is rejected.
- Storage error → System prompts: *“Upload failed. Try again.”*

FR-3: Resource Access (Browse, View, Download)

Description:

The system must allow all verified users to access shared resources and interact with them.

Scenario:

1. The user opens the **Resources** page.
2. The system displays a list of resources.
3. The user selects a resource to view detailed information.
4. The user clicks **Download, or view**.

Exceptions:

- Resource removed by admin → System displays: *“This resource is no longer available.”*
- File missing in storage → System notifies admin automatically.

FR-4: Search and Filter

Description:

The system must allow users to search and filter resources based on keywords and metadata.

Scenario:

1. The user enters keywords in the search bar.
2. The user applies filters (subject, university).

3. The system retrieves matching resources.
4. The user selects a resource from the results list.

Exceptions:

- No matching results → System displays: “*No resources found.*”

FR-5: Chat System

Description:

The system must provide messaging functionality for group channels.

Scenario:

1. The user opens the **Chat** section.
2. The system loads group channels.
3. The user selects a conversation.
4. The user types a message and sends it.
5. The system stores the message and delivers it to recipients.

Exceptions:

- Sending an empty message → System rejects input.

FR-6: Help Center (Contact Support)

Description:

The system shall allow users to send support inquiries via email.

Admins shall respond externally via their admin mailbox.

Notes:

- No database table is required.
- No ticket status tracking exists in the system.

Scenario:

1. The user opens the **Help Center**.

2. The user clicks “**Contact Us.**”
3. The user sends the email.

Exceptions:

- User not logged in → The system redirects to the login page with the message: “You must be logged in to contact support.”

FR-7: Notifications

Description:

The system must notify users of relevant events such as replies.

Scenario:

1. A trigger event occurs (e.g., report resolved).
2. The system generates a notification.
3. The user opens the notification panel.
4. The system displays unread and read notifications.
5. The user clicks a notification to open the related page.

Exceptions:

- Notification fails to load → System displays the fallback message: “*Unable to retrieve notifications.*”

3.2 External Interface Requirements

Type	Description
Hardware	User device with internet browser.
Software Interfaces	REST API between frontend and backend; PostgreSQL for data storage.
Communication Interfaces	Communication Interfaces:

	- HTTPS (TLS 1.2+) for all client–server REST API communication - WSS (Secure WebSocket) for real-time chat messaging
--	--

3.3 Performance Requirements

- Response time ≤ 3 seconds for page loads.
- Chat message latency ≤ 1 second.
- Support ≥ 500 concurrent users without downtime.
- Uptime $\geq 99.5\%$.

3.4 Logical Database Requirements

- Separate tables for users, universities, resources, and reports.
- Referential integrity between users and uploads.
- Automatic backup every 24 hours; restore within 1 hour.

3.5 Design Constraints

- Technologies: React.js + Tailwind CSS for the frontend,
- Python (Flask or Django) for the backend, and PostgreSQL as the database system.
- All REST API communications are encrypted via HTTPS.
- Real-time chat communication is encrypted via WSS (Secure WebSockets).
- Code follows modular MVC architecture.

3.6 Security Requirements

- Encrypted password storage (bcrypt).
- All REST API communication encrypted via HTTPS.
- Role-based authorization system.
- GDPR-compliant data retention policy.

3.7 Reliability and Availability Requirements

- 99.5% uptime target.
- Automatic recovery from server crashes.
- Monitoring with alerts for critical failures.

3.8 Maintainability and Portability Requirements

- Modular code structure with clear APIs.
- Portable across major OS and browsers.
- Compatible across Windows, macOS, and Linux browsers.

3.9 Non-Functional Requirements

Category	Requirement
Usability	UI must remain consistent and intuitive across all pages.
Scalability	Expandable to 2,000 users by adding servers.
Accessibility	Meets WCAG 2.1 Level AA standards.
Legal	Must comply with GDPR standards.
Ethical	Users agree to Community Guidelines upon registration.

4 Appendices

4.1 Glossary

Term	Definition
Resource	Any academic file (PDF, DOCX) uploaded by a student.

Report	A user-submitted email for inappropriate or problematic content.
JWT	JSON Web Token used for secure authentication.
GDPR	EU data protection regulation defining privacy and data-handling rules.
REST API	Architectural style for communication between backend and frontend.

4.2 Data Dictionary

This data dictionary defines the key entities, attributes, and constraints used in the logical database design for EduCampus.

4.2.1 Universities

Attribute	Type	Description	Constraints
university_id	INT	Unique identifier	PK, auto-increment
name	VARCHAR(150)	University name	Not null, unique
city	VARCHAR(100)	City (e.g., Potsdam)	Not null
country	VARCHAR(100)	Country	Not null
domain	VARCHAR(150)	Email domain	Not null, unique
created_at	TIMESTAMP	When record was added	Default now()

4.2.2 Users

Attribute	Type	Description	Constraints
user_id	INT	Unique user ID	PK, auto-increment
university_id	INT	Related university	FK → universities
role	VARCHAR(20)	student	Not null
first_name	VARCHAR(100)	First name	Not null
last_name	VARCHAR(100)	Last name	Not null
email	VARCHAR(255)	University email	Not null, unique
password_hash	VARCHAR(255)	Hashed password	Not null
profile_bio	TEXT	Optional bio	Nullable
created_at	TIMESTAMP	Registration date	Default now()
last_login	TIMESTAMP	Last login	Nullable

4.2.3 Resources

Attribute	Type	Description	Constraints
resource_id	INT	Resource ID	PK, auto-increment
uploader_id	INT	Student who uploaded	FK → users

university_id	INT	University ownership	FK → universities
title	VARCHAR(200)	Resource title	Not null
description	TEXT	Summary	Nullable
subject	VARCHAR(100)	Subject/topic	Not null
file_url	VARCHAR(500)	File location	Not null
file_type	VARCHAR(20)	PDF/DOCX	Not null
file_size_mb	NUMERIC(6,2)	≤ 100 MB	Check constraint
created_at	TIMESTAMP	Upload time	Default now()
is_active	BOOLEAN	Visible or removed	Default: true

4.2.4 Messages (Chat)

Attribute	Type	Description	Constraints
message_id	INT	Message ID	PK
sender_id	INT	Sender	FK → users
receiver_id	INT	Receiver	FK (nullable)
channel_id	INT	Group channel ID	Nullable

content	TEXT	Message	Not null
created_at	TIMESTAMP	Sent time	Default now()
is_read	BOOLEAN	Read status	Default false

4.2.5 Notifications

Attribute	Type	Description	Constraints
notification_id	INT	Unique ID	PK
user_id	INT	Receiver	FK
type	VARCHAR(50)	Notification type	Not null
payload	JSON	Metadata	Nullable
is_read	BOOLEAN	Read flag	Default false
created_at	TIMESTAMP	Created	Default now()

4.3 Sample Data

4.3.1 Universities

ID	Name	City	Domain
1	University of Potsdam	Potsdam	uni-potsdam.de

2	UE University of Europe	Potsdam	ue-germany.com
3	Fachhochschule Potsdam	Potsdam	fh-potsdam.de

4.3.2 Users

ID	University	Role	Name	Email
1	UE	Student	Lara Hoffmann	lara.h@ue-germany.com
2	Uni Potsdam	Student	Jonas Weber	jonas.w@uni-potsdam.de
3	UE	Admin	Admin EduCampus	admin@educampus.de

4.3.3 Resources

ID	Title	Uploader	Subject
10	SE2 – SRS Notes	Lara	Software Engineering
11	Linear Algebra Summary	Jonas	Mathematics

4.4 Mockups

This link gives access to the prototype for EduCampus.

<https://kilt-cot-83401268.figma.site/>

Summary

The SRS specifies **EduCampus as a student-to-student collaboration platform** designed to enhance academic exchange within Potsdam's university ecosystem.

It defines measurable requirements for secure registration, P2P file sharing, moderated interaction, and admin-managed support ensuring compliance with IEEE standards and readiness for academic evaluation.